Jantar dos Filósofos

Resumo do código para estudo e apresentação • Java e concorrência

O que o programa representa?

Cinco filósofos alternam entre pensar e comer em uma mesa circular. Cada um precisa dos dois garfos ao seu lado para comer. No código, cada filósofo executa em uma **thread** e cada garfo é um **ReentrantLock**, que permite uso exclusivo do recurso. O exemplo demonstra como coordenar tarefas que disputam recursos compartilhados.

Responsabilidade de cada classe

Classe	Responsabilidade
App	Cria uma única mesa, inicia cinco filósofos e aguarda suas threads com join().
Table	Cria e armazena os cinco bloqueios que representam os garfos.
Philosopher	Implementa Runnable e repete: pensar, adquirir dois garfos, comer e liberar.

Como funciona a execução

1. App compartilha a mesma Table entre os cinco filósofos.
2. start() inicia cada thread, que executa run(). O join() apenas aguarda: não executa os filósofos um por vez.
3. Cada filósofo pensa por um intervalo aleatório de 0 a 4.999 ms.
4. Adquire os dois garfos, come por 0 a 9.999 ms e os libera em blocos finally.
5. O ciclo se repete indefinidamente. A ordem das mensagens varia entre execuções.

A regra que evita deadlock

Um **deadlock** ocorreria se cada filósofo segurasse um garfo e esperasse pelo garfo ocupado pelo vizinho, formando um círculo sem progresso. A solução adquire sempre o garfo de **menor índice primeiro**, usando Math.min(), e depois o de maior índice, usando Math.max().

Ordens de aquisição: **F0: 0 → 1 | F1: 1 → 2 | F2: 2 → 3 | F3: 3 → 4 | F4: 0 → 4.** Essa ordem global impede a espera circular entre os garfos. A aleatoriedade não é responsável por evitar o deadlock.

O que observar na demonstração

Filósofos vizinhos não comem juntos. Filósofos que usam garfos diferentes podem comer simultaneamente: por exemplo, 0 e 2. Com cinco garfos, **no máximo dois** comem ao mesmo tempo. sleep() não libera bloqueios; os finally executam os unlock(). A mensagem “está comendo” indica que os dois garfos foram adquiridos.

Limitações importantes

Justiça: os bloqueios padrão não garantem que todos tenham a mesma oportunidade; pode ocorrer inanição (starvation). Evitar deadlock não garante ausência de starvation.
Interrupção: lock() não cancela imediatamente a espera ao receber interrupt(); lockInterruptibly() seria uma opção para evoluir o cancelamento.
Encerramento: não há término automático nem parada coordenada de todas as threads.
Quantidade fixa: filósofos e garfos estão definidos como cinco em lugares distintos; mudar apenas um deles pode causar erros.

Mensagem principal para a apresentação

O projeto demonstra exclusão mútua e prevenção de deadlock por ordenação de recursos. App coordena a inicialização, Table representa os recursos compartilhados e Philosopher executa o comportamento concorrente. A solução evita espera circular, mas não garante justiça.